Introduction

CSE 100/1000/1100: Introduction to Computer Engineering

Instructor:
Fatma CORUT ERGİN

Slides adapted from

- *McGraw Hill Companies, and*
- *Walid A. Najjar & Brian J. Linard*

slides

This course is about:

- What computers consist of
- How computers work
- How they are organized internally
- What are the design tradeoffs
- How design affects programming and applications

- How to fix computers
- How to build myself one real cheap
- Which one to buy
- Knowing all about the Pentium IV or PowerPC

Computing Machines

Ubiquitous (= everywhere)

- General purpose: servers, desktops, laptops, PDAs, etc.
- Special purpose: cash registers, ATMs, games, telephone switches, etc.
- Embedded: cars, hotel doors, printers, VCRs, industrial machinery, medical equipment, etc.

Distinguishing Characteristics

- Speed
- Cost
- Ease of use, software support & interface
- Scalability

Two recurring themes

Abstraction

- **The notion that we can concentrate on one "level" of the big picture at a time, with confidence that we can then connect effectively with the levels above and below.**
- **Framing the levels of abstraction appropriately is one of the most important skills in *any* undertaking.**

Hardware vs. Software

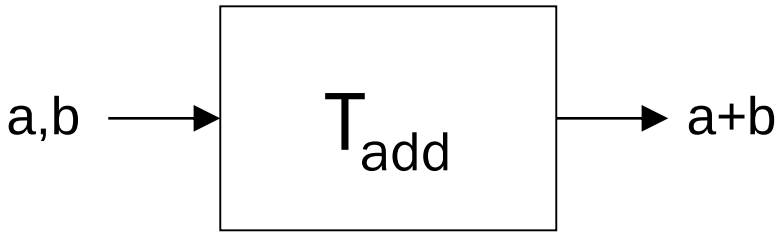
- **On the other hand, abstraction does *not* mean being clueless about the neighboring levels.**
- **In particular, hardware and software are inseparably connected, especially at the level we will be studying.**

Turing Machine

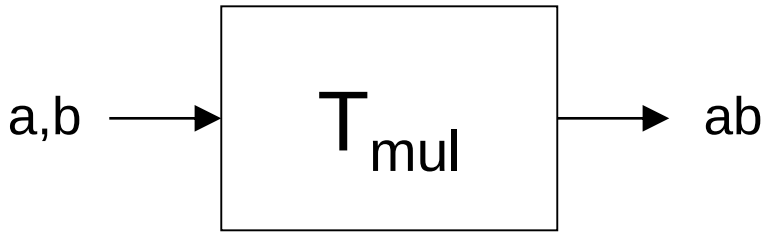
Mathematical model of a device that can perform any computation – Alan Turing (1937)

- **ability to read/write symbols on an infinite "tape"**
- **state transitions, based on current state and symbol**

Every computation can be performed by some Turing machine. ***(Turing's thesis)***



Turing machine that adds



Turing machine that multiplies

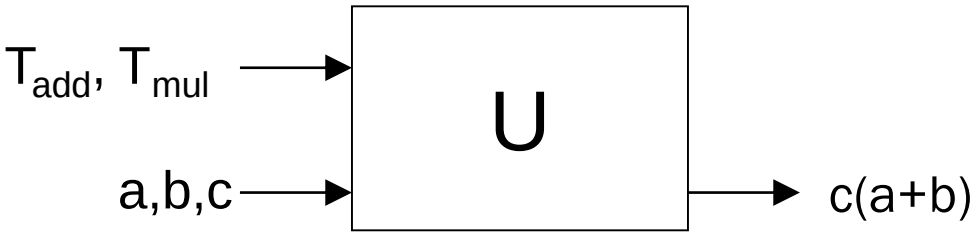
For more info about Turing machines, see http://www.wikipedia.org/wiki/Turing_machine/

For more about Alan Turing, see http://www.turing.org.uk/turing/

Universal Turing Machine

A machine that can implement all Turing machines -- this is also a Turing machine!

- **inputs: data, plus a description of computation (other TMs)**



Universal Turing Machine

U is programmable – so is a computer!

- **instructions are part of the input data**
- **a computer can emulate a Universal Turing Machine**

A computer is a universal computing device.

From Theory to Practice

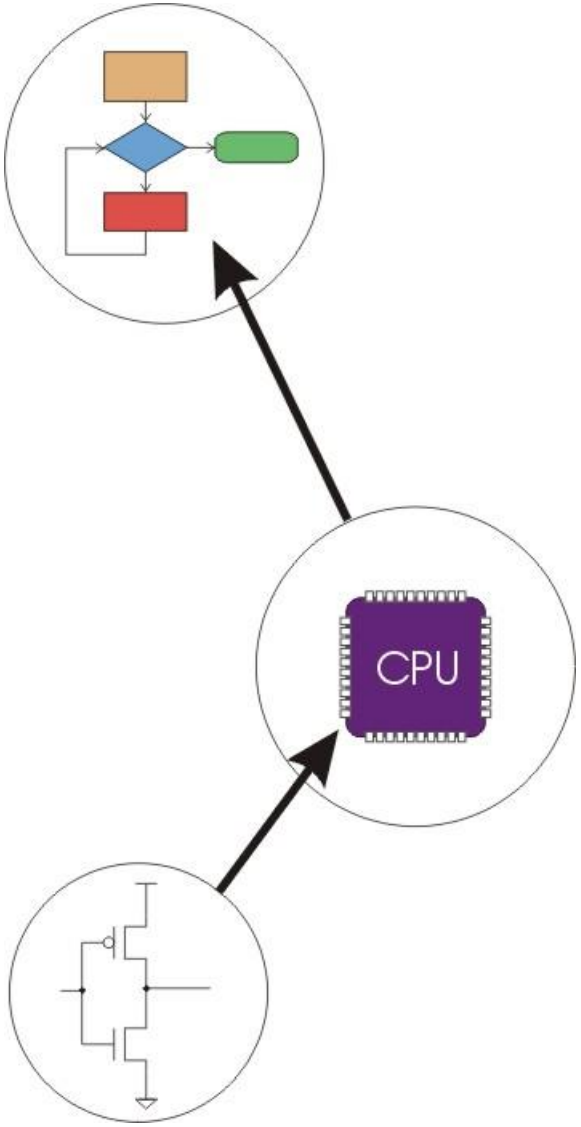
In theory, computer can *compute* anything that's possible to compute

- **given enough *memory* and *time***

In practice, *solving problems* involves computing under constraints.

- **time**
  - **weather forecast, next frame of animation, ...**
- **cost**
  - **cell phone, automotive engine controller, ...**
- **power**
  - **cell phone, handheld video game, ...**

Big Idea #2: Transformations Between Layers



Problems

Algorithms

Language

Instruction Set Architecture

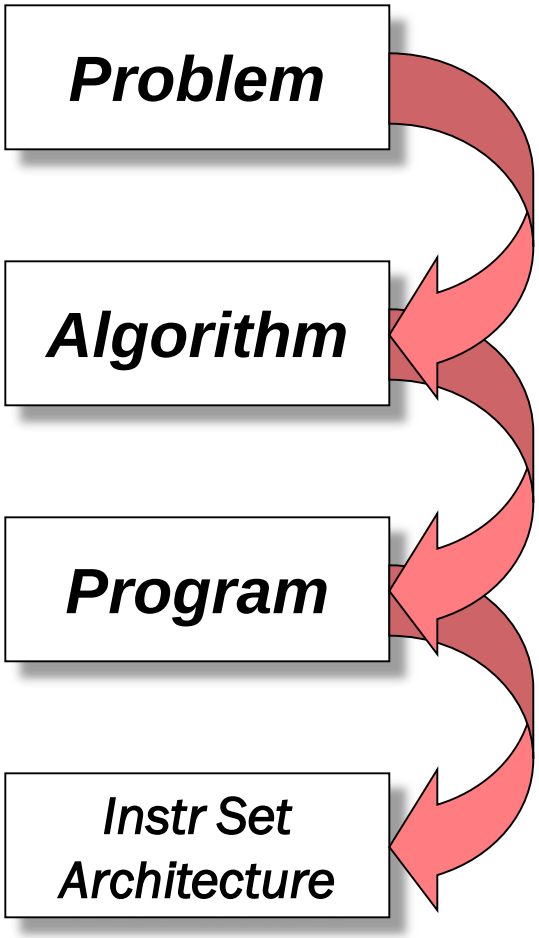
Microarchitecture

Circuits

Devices

How do we solve a problem using a computer?

A systematic sequence of transformations between layers of abstraction.

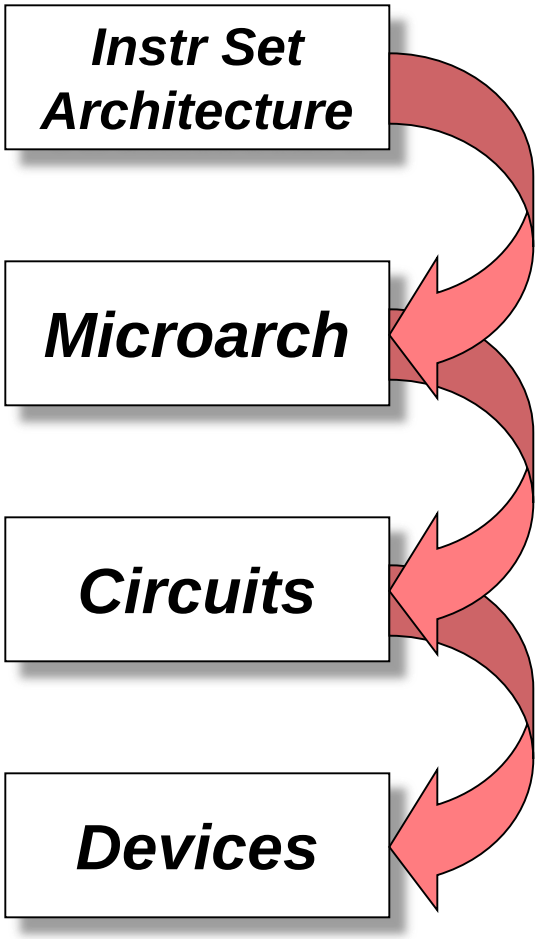


Software Design:
choose algorithms and data structures

Programming:
use language to express design

Compiling/Interpreting:
convert language to machine instructions

Deeper and Deeper…



Processor Design:
choose structures to implement ISA

Logic/Circuit Design:
gates and low-level circuits to implement components

Process Engineering & Fabrication:
develop and manufacture lowest-level components

Descriptions of Each Level

Problem Statement

- stated using "natural language"
- may be ambiguous, imprecise

Algorithm

- step-by-step procedure, guaranteed to finish
- definiteness, effective computability, finiteness

Program

- express the algorithm using a computer language
- high-level language, low-level language

Instruction Set Architecture (ISA)

- specifies the set of instructions the computer can perform
- data types, addressing mode

Descriptions of Each Level (cont.)

Microarchitecture

- **detailed organization of a processor implementation**
- **different implementations of a single ISA**

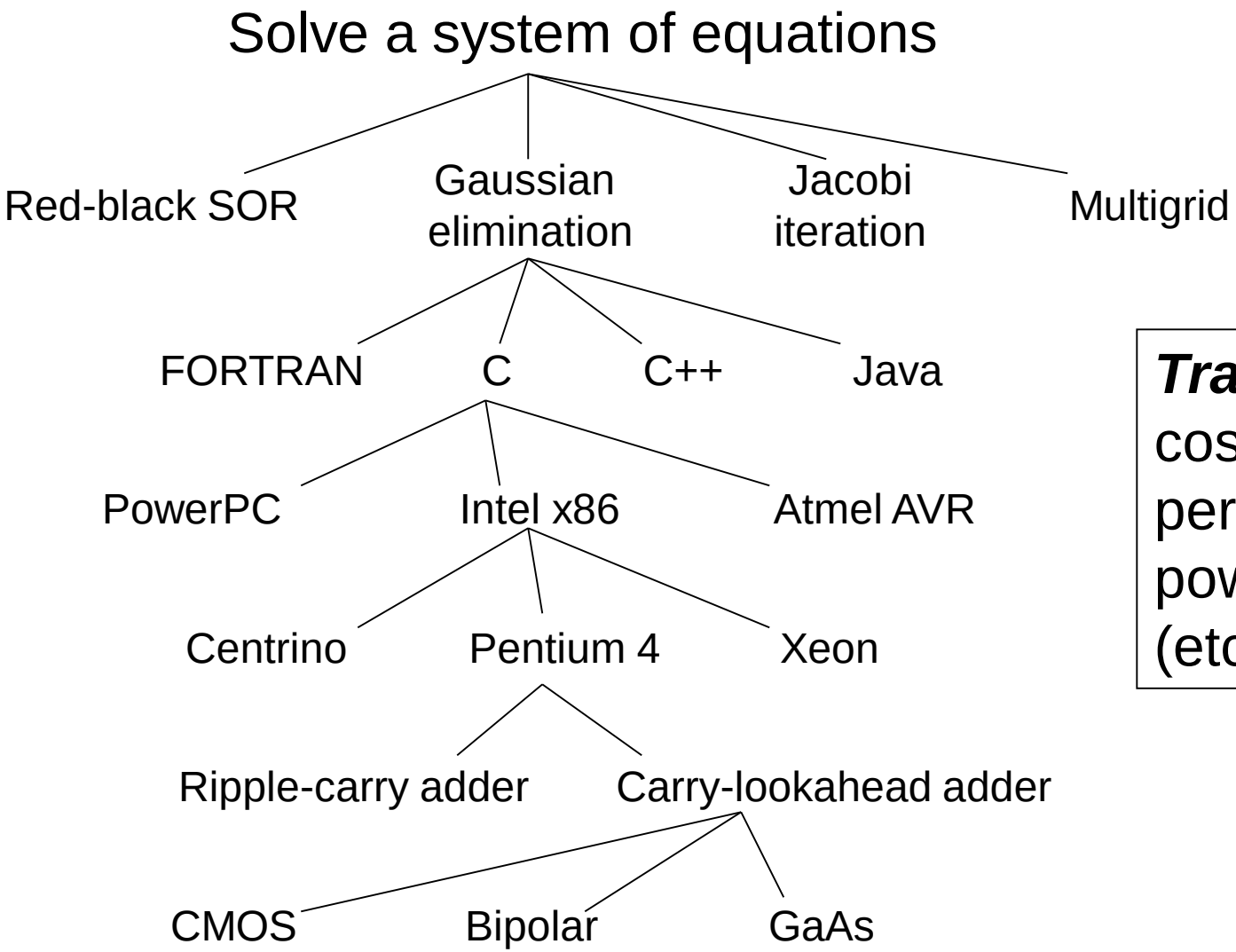
Logic Circuits

- **combine basic operations to realize microarchitecture**
- **many different ways to implement a single function (e.g., addition)**

Devices

- **properties of materials, manufacturability**

Many Choices at Each Level



Tradeoffs:
cost
performance
power
(etc.)

Course Outline

Bits and Bytes

- How do we represent information using electrical signals?

Digital Logic

- How do we build circuits to process information?

Processor and Instruction Set

- How do we build a processor out of logic elements?
- What operations (instructions) will we implement?

Assembly Language Programming

- How do we use processor instructions to implement algorithms?
- How do we write modular, reusable code? (subroutines)

I/O, Traps, and Interrupts

- How does processor communicate with outside world?